

# 클라우드 기반 지능형 스마트 팩토리 통합 관제 시스템

컴퓨터인공지능학부  
202017679  
신서현

# 목차

1 프로젝트 개요

---

2 시스템 설계

---

3 주요 기능

---

chapter 1

---

# 프로젝트 개요

문제 인식

개발 목적

프로젝트 소개

# 제조 현장의 디지털 전환과 한계

## AI·스마트팩토리 전환 가속...4대 그룹, 제조 혁신 본격화

이선율 기자

입력 2025.12.10 13:59

삼성전자와 SK그룹, LG, 현대차 등 주요 4대 그룹이 제조 공장을 'AI가 스스로 판단하는 공장'으로 전환하는 작업에 속도를 올리고 있다. 반도체·가전·자동차 등 전 사업군에서 GPU와 디지털트윈 그리고 로봇틱스 기술을 생산라인에 적용하며 기존 자동화 수준을 뛰어넘는 AI 팩토리 구축 경쟁이 본격화하고 있다.



출처 : 연합뉴스 (2024.06)

스마트 팩토리 전환 가속화

스마트 팩토리도입으로  
초당 수천 개의 IoT 센서 데이터 발생

1. 트래픽 폭주 시 서버 다운
2. 장애 발생 시 수동 복구의 한계
3. 데이터는 쌓이지만 분석되지 않는 비효율



Loose Coupling, Serverless, Fault Tolerance

1

## 실시간 통합 관제

데이터의 흐름과  
시스템 건강 상태를  
한눈에 파악

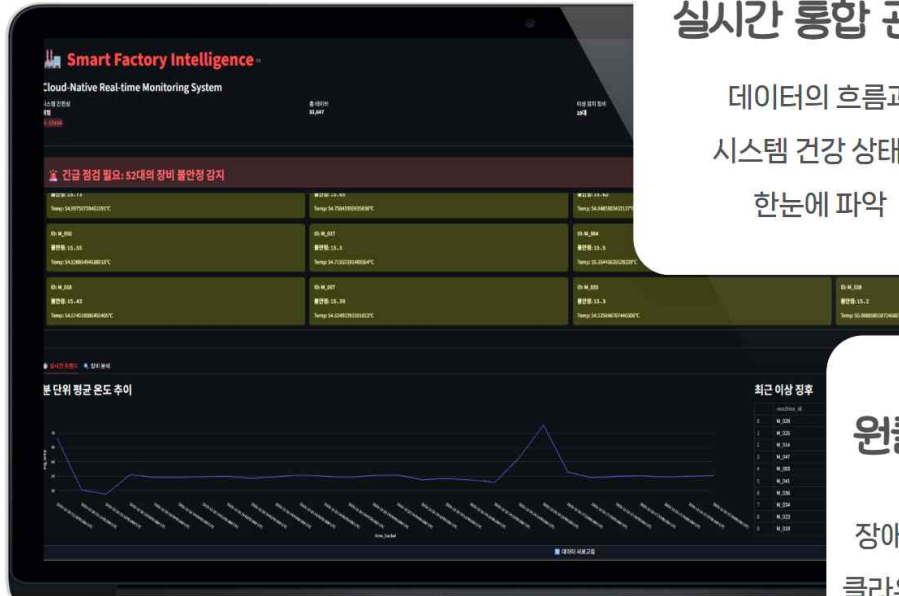
2

지능형  
예지 보전 알림  
통계적 분석을 통한  
고장 위험 높은 장비  
선제적 경고

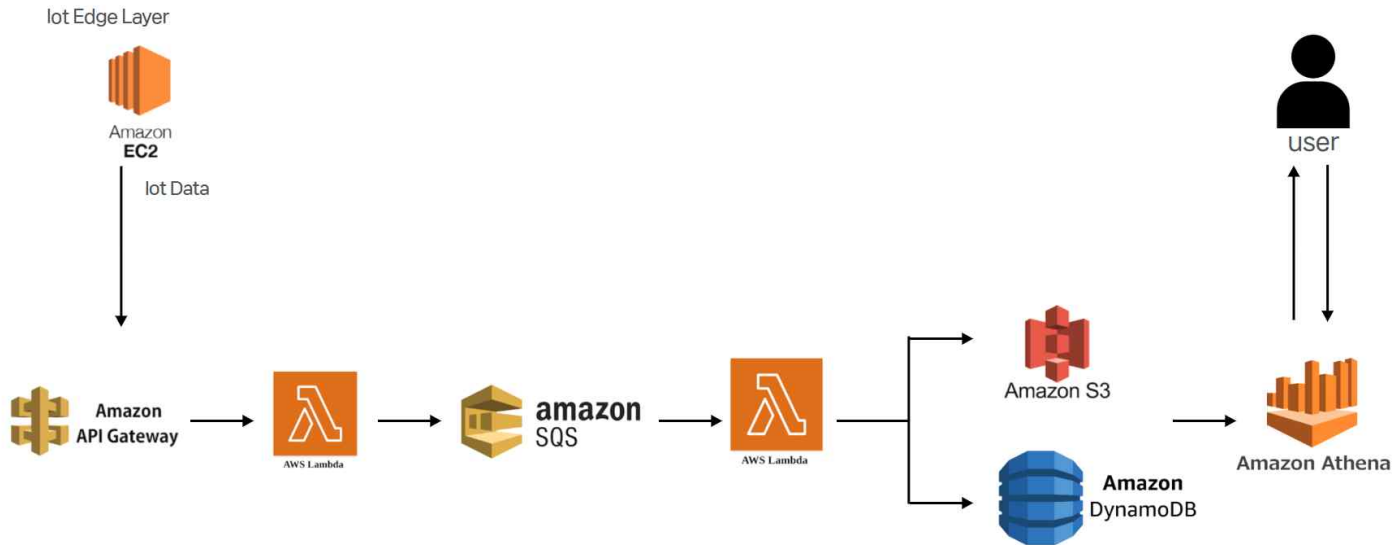
3

## 원클릭 인프라 제어

장애 발생 시 사용자가 직접  
클라우드 복구 로직 실행 가능



## 초기 아키텍처



## IoT 시뮬레이터

```
# [실험 변수] 이 수치를 조절하며 실험합니다.
NUM_DEVICES = 1000      # 기계 대수를 확장
INTERVAL = 0.01         # 전송 간격 (부하 증가 테스트 때 사용)
ERROR_RATE = 0.05       # 5% 확률로 고장난 데이터(DLQ용) 발송
ANOMALY_RATE = 0.1      # 10% 확률로 이상치(80도 이상) 발송
```

```
def send_data(device_id):
    # 1. 에러/이상치/정상 데이터 생성 로직
    rand_val = random.random()

    if rand_val < ERROR_RATE:
        # [시나리오] 센서 고장으로 인한 문자열 데이터 (DLQ 유도)
        temp_value = "SENSOR_ERROR"
    elif rand_val < (ERROR_RATE + ANOMALY_RATE):
        # [시나리오] 실제 기계 과열 (이상 탐지 로그 유도)
        temp_value = round(random.uniform(85, 110), 2)
    else:
        # [시나리오] 정상 가동
        temp_value = round(random.uniform(40, 60), 2)

    payload = {
        "event_id": str(uuid.uuid4()),
        "timestamp": datetime.datetime.now().isoformat(),
        "factory_id": "SEOUL_FACT_01",
        "machine_id": device_id,
        "sensor_type": "temperature",
```

## 실행 결과

```
• (venv) PS C:\Users\seohyeon\gitProject\cloudComputin
g> & C:/Users/seohyeon/gitProject/cloudComputing/ven
v/Scripts/python.exe c:/Users/seohyeon/gitProject/cl
oudComputing/simulator.py
Sent: M_05 | Temp: 57.36 | Status: 200
Sent: M_04 | Temp: 48.55 | Status: 200
Sent: M_01 | Temp: 46.94 | Status: 200
Sent: M_02 | Temp: 59.16 | Status: 200
```



## API Gateway

- 외부 시뮬레이터(공장 역할)를 클라우드로 연결하는 입구 기능
- 보안: API Key 또는 IAM 권한을 통한 접근 제어
- RESTful 엔드포인트 구축: 외부 IoT 기기(시뮬레이터)와의 통신을 위한 인터페이스 표준화
- 추상화: 뒤쪽의 SQS나 Lambda가 바뀌어도 앞단의 엔드포인트는 유지하여 유지보수성 확보

Successfully created API SmartFactoryAPI (wnfb6kkl9c).

### 경로

스테이지: - ▼

배포

#### SmartFactoryAPI에 대한 경로

생성

Q 검색

▼ /ingest

POST

## SmartFactoryAPI

스테이지: - ▼

배포

### API 세부 정보

편집

API ID

wnfb6kkl9c

프로토콜

HTTP

생성됨

2025-12-20

설명

No Description

IP 주소 유형

-

기본 엔드포인트

✓ 활성화됨

<https://wnfb6kkl9c.execute-api.ap-northeast-2.amazonaws.com>

ARN

<arn:aws:apigateway:ap-northeast-2::apis/wnfb6kkl9c>

### SmartFactoryAPI에 대한 스테이지 (1)

Q 리소스 찾기

스테이지 이름

URL 호출

[\\$default](#)

<https://wnfb6kkl9c.execute-api.ap-northeast-2.amazonaws.com>

## SQS

- Decoupling: API 수집 속도와 Lambda 처리 속도가 달라도 시스템이 멈추지 않게 함
- Availability: 3개의 가용 영역(AZ)에 메시지 자동 복제
- Visibility Timeout: 람다가 처리 중인 메시지를 다른 인스턴스가 건드리지 못하게 보호
- Durability : 람다에 일시적인 장애가 생겨도 데이터가 즉시 삭제되지 않고 큐에 안전하게 보관됨

## 대기열 (2)



편집

삭제

메시지 전송 및 수신

작업 ▼

대기열 생성

Q 접두사를 기준으로 대기열 검색

&lt; 1 &gt;



|                       | 이름 ▲                               | 유형 ▼ | 생성 날짜 ▼                | 사용 가능한 메시지 ▼ | 이동 중인 메시지 ▼ | 암호 |
|-----------------------|------------------------------------|------|------------------------|--------------|-------------|----|
| <input type="radio"/> | <a href="#">factory-data-dlq</a>   | 표준   | 2025-12-20T20:42+09:00 | 507          | 0           | Am |
| <input type="radio"/> | <a href="#">factory-data-queue</a> | 표준   | 2025-12-20T20:44+09:00 | 7106         | 730         | Am |

factory-data-dlq : 제대로 처리되지 못한 메시지 처리  
 factory-data-queue : 전송된 메시지 처리

## SQS vs Kinesis

| 비교 항목  | SQS       | Kinesis         |
|--------|-----------|-----------------|
| 모델     | 58        | 51              |
| 데이터 처리 | 59        | 52              |
| 순서 보장  | 60        | 61              |
| 확장성    | 거의 무제한    | 샤드 개수에 따라 확장 필요 |
| 비용     | 사용한 만큼 지불 | 시간당 샤드 유지 비용 발생 |
| 복잡도    | 연동 쉬움     | 샤드 관리 및 설정 필요   |

## SQS vs Kinesis

## kinesis 구축 실험

## 실험 설정

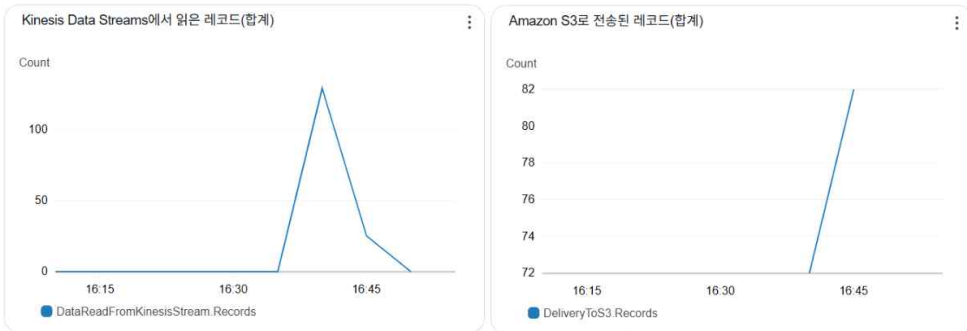
EPS: 1000  
Duration: 120초  
Shard 수: 1  
Workers: 128

"정상 운영 상황에서의 기준 성능"

```
python simulator.py --mode http --api-url https://93trnak1f6.execute-api.us-east-1.amazonaws.com/prod/ingest --duration-s 120 --target-eps 200 -anomaly-rate 0.002 --duplicate-rate 0.02object\cloudComputin
>>
[RESULT] sent=154 ok=154 failed=0 dup=4 seconds=121.62
[RESULT] achieved_eps=1.3
(venv) PS C:\Users\seohyeon\gitProject\cloudComputing>
```



Kinesis Data Streams의  
PutRecord 지표를 통해  
Lambda가 스트림으로  
이벤트를 지속점으로 전달함을  
확인했습니다.



Firehose를 통해  
S3로 데이터가 잘 이동하는 것을  
확인했습니다.

07/

S3 URI 복사

객체

속성

객체 (3)









객체는 Amazon S3에 저장되어 있는 기본 엔티티입니다. [Amazon S3 인벤토리](#)를 사용하여 버킷에 있는 모든 객체의 목록을 얻을 수 있습니다. 다른 사용자가 객체에 액세스할 수 있게 하려면 명시적으로 권한을 부여해야 합니다. [자세히 알아보기](#)

&lt; 1 &gt;


☐ 이름 ▲ | 유형 ▼ | 마지막 수정 ▼ | 크기 ▼ | 스토리지 클래스 ▼

| 이름                                                  | 유형 | 마지막 수정                      | 크기     | 스토리지 클래스 |
|-----------------------------------------------------|----|-----------------------------|--------|----------|
| cc-smartfactory-firehose-raw-1-2025-12-20-07-43-18- | -  | 2025. 12. 20. pm 4:44:22 PM | 34.5KB | Standard |

S3를 보면 파일 단위로 잘 적재되었음을 확인할 수 있습니다.

정상 부하 환경에서 서버리스 스트리밍 파이프라인의 안정성과 데이터 전달 흐름을 검증하였습니다.

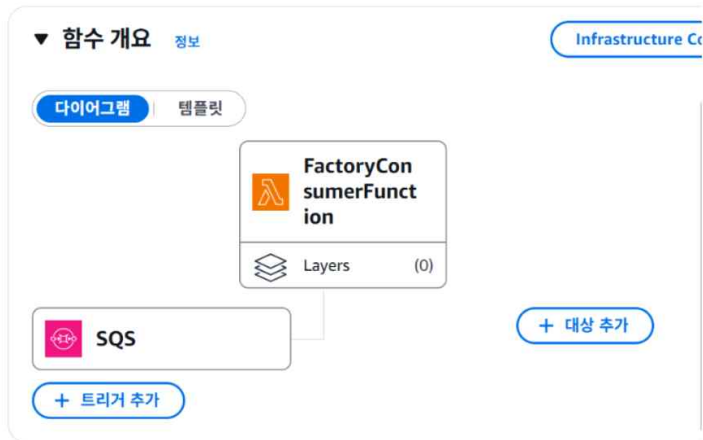
API Gateway-Lambda-Kinesis-Firehose-S3로 이어지는 전체 파이프라인에서 오류 없이 데이터가 처리됨을 확인하였습니다.

하지만, 아래와 같은 이유때문에 이번 프로젝트에서는 SQS를 선택했습니다.

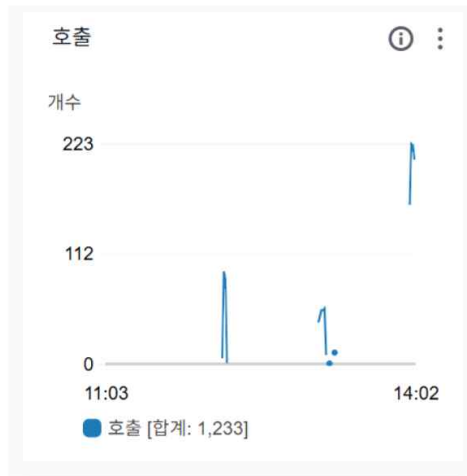
- 비용 효율성 : 초기 단계에서 SQS는 프리티어 범위 내에서 거의 무료이지만, Kinesis는 사용량과 관계없이 샤드 유지비가 발생합니다. 학생 프로젝트로서 자원 낭비를 최소화하였습니다. 사실 이 이유가 가장 큽니다.
- 운영 간소화 : SQS는 관리형 서비스로 확장이 자동이지만, Kinesis는 트래픽에 따라 샤드를 늘리고 줄이는 관리 포인트가 발생합니다. 저는 이상치 탐지에 더 집중하기 위해 SQS를 선택했습니다.
- 데이터 성격: 현재 스마트 팩토리 시나리오는 메시지 재처리보다는 발생 즉시 판별이 중요하므로 큐 방식이 더 적합하다고 생각했습니다.

## Lambda

"Event-Driven 아키텍처: SQS 트리거를 통한 비동기식 데이터 처리 및 실시간 이상 탐지 로직 구현."



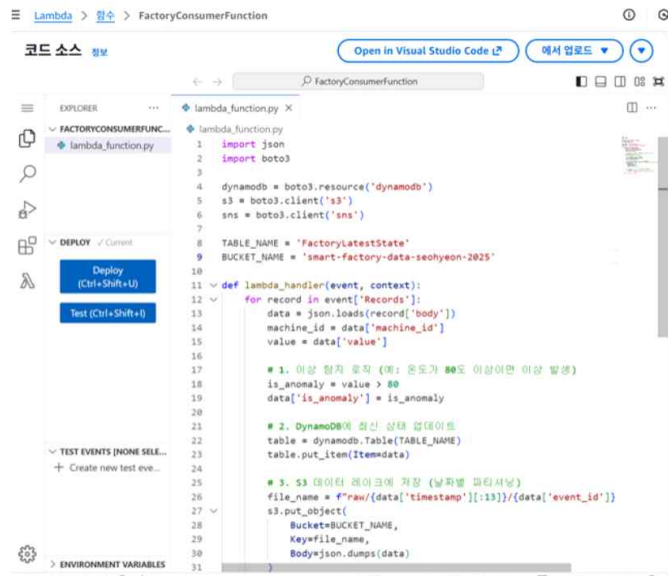
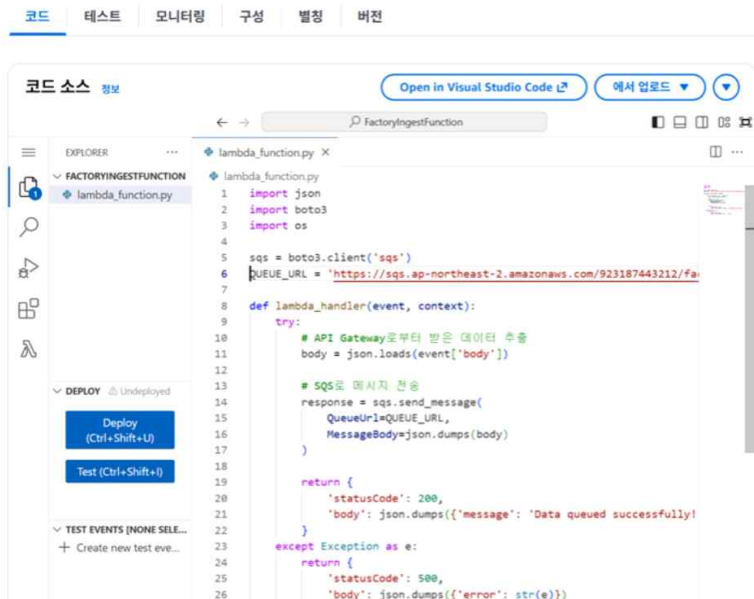
lambda의 호출 그래프



# Lambda

ingest 레이어 구현 : 외부(EC2 시뮬레이터)에서 들어오는 데이터를 일단 받아서 SQS로 던지는 역할. api gateway와 sqs 사이의 느슨한 결합을 구현하여 가용성 확보

이상 탐지 전용 lambda : SQS에 데이터가 들어오면 자동으로 실행되어 임계치를 넘는 데이터들을 골라내서 dynamoDB/s3로 보냄





# Dynamo DB / S3

데이터 영구 보관 참고 (s3) : 원본 데이터는 비용이 저렴한  
s3 데이터 레이크에 저장

s3 버킷 내부에 시간/날짜별로 폴더가 생성됨

Amazon S3 > 버킷 > smart-factory-data-seohyeon-2025 > raw/ > 2025-12-20T13/

2025-12-20T13/

S3 URI 복사

객체 속성

객체 (231)

S3 URI 복사 URL 복사 다운로드 열기 내 삭제 작업 더보기

없음

객체는 Amazon S3에 저장되어 있는 기본 엔티티입니다. Amazon S3 인벤토리 콘솔을 사용하여 버킷에 있는 모든 객체의 목록을 얻을 수 있습니다. 다른 사용자가 객체에 액세스할 수 있게 하려면 명시적으로 권한을 부여해야 합니다. 자세한 알아보기

Q. 전두사로 객체 찾기

< 1 >

| 이름                                        | 유형   | 마지막 수정                           | 크기     | 스토리지 클래스 |
|-------------------------------------------|------|----------------------------------|--------|----------|
| 012d0835-a8a1-440e-b0c8-a078f97d510d.json | json | 2025. 12. 20. pm 10:07:11 PM KST | 213.0B | Standard |
| 022c2049-d8b6-460f-923f-6045e43a9385.json | json | 2025. 12. 20. pm 10:10:07 PM KST | 212.0B | Standard |
| 02320b40-4be2-4f66-8f72-7b92e46e8862.json | json | 2025. 12. 20. pm 10:07:06 PM KST | 213.0B | Standard |
| 033d385f-0be1-4273-992d-4f6c820ae5ac.json | json | 2025. 12. 20. pm 10:08:49 PM KST | 212.0B | Standard |
| 03ad90ec-6c4b-4a2b-812b-1752e7ae10e9.json | json | 2025. 12. 20. pm 10:09:50 PM KST | 213.0B | Standard |

dynamo db에 기계별 최신 데이터가 들어와있음  
실시간 현황판 (DynamoDB) : 최근 상태는 빠른 조회가  
가능한 db에 저장

FactoryLatestState

항목 스캔 또는 쿼리

스캔

쿼리

데이터를 또는 인덱스 선택

데이터 - FactoryLatestState

속성 프로jections 선택

모든 속성

필터 - 선택 사항

실행

재실행

완료됨. 반환된 항목: 5 - 스캔한 항목: 5 - 효율성: 100% - 소비된 RCU: 2

데이터: FactoryLatestState - 반환된 항목 (5)

스캔 시작 날짜: 12월 20, 2025, 22:19:37

| machine_id (문자열) | event_id      | factory_id   | is_anomaly | sensor_type | timestamp    | unit    | value |
|------------------|---------------|--------------|------------|-------------|--------------|---------|-------|
| M_01             | 4298f422-...  | SEOUL_FAC... | false      | temperature | 2025-12-2... | Celsius | 46.33 |
| M_02             | 7c95fd71-d... | SEOUL_FAC... | false      | temperature | 2025-12-2... | Celsius | 53.41 |
| M_04             | 09150763-...  | SEOUL_FAC... | false      | temperature | 2025-12-2... | Celsius | 48.49 |

## Athena

## 서버리스 데이터 레이크 분석

쿼리를 이용해서 실시간 데이터는 dynamoDB 에서  
 예측할땐 S3에서 대용량 통계, 과거 이력 분석, 표준 편차를 계산해서 고장 징후를 찾아 데이터를 분석했습니다.



Amazon  
DynamoDB



Amazon S3

```
# 3. 예지 보전 섹션 (Athena 통계 활용)
st.divider()
unstable_targets = vol_df[vol_df['instability'] > INSTABILITY_THRES]
if not unstable_targets.empty:
    st.error(f"## 🚨 분석 결과: {len(unstable_targets)}대의 장비에서 감
    with st.container(height=350):
        cols_per_row = 4
        for i in range(0, len(unstable_targets), cols_per_row):
            row_targets = unstable_targets.iloc[i : i + cols_per_row]
            cols = st.columns(cols_per_row)
            for j, (_, row) in enumerate(row_targets.iterrows()):
                with cols[j]:
                    st.warning(f"**ID: {row['machine_id']}**\n\n불온
```



Amazon Athena

## Athena

예시) 표준편차를 이용한 예지 보전 로직  
고장 나기 직전의 기계를 표준편차를 이용해서 미리 찾을 수 있음

## 쿼리

쿼리 1 :

( + ) ▼

```

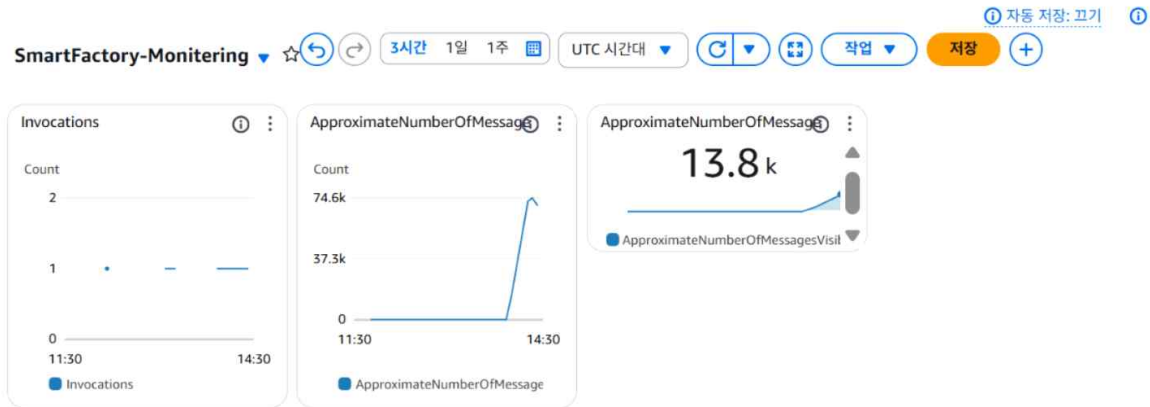
1  SELECT
2      machine_id,
3      ROUND(STDDEV(value), 2) AS instability, -- 온도의 표준편차 (불안정 지수)
4      ROUND(AVG(value), 2) AS avg_value,      -- 평균 온도
5      COUNT(*) AS record_count               -- 수집된 데이터 개수
6  FROM
7      "factory_db"."sensor_data"
8  GROUP BY
9      machine_id
10 ORDER BY
11     instability DESC;
```

## 실행결과

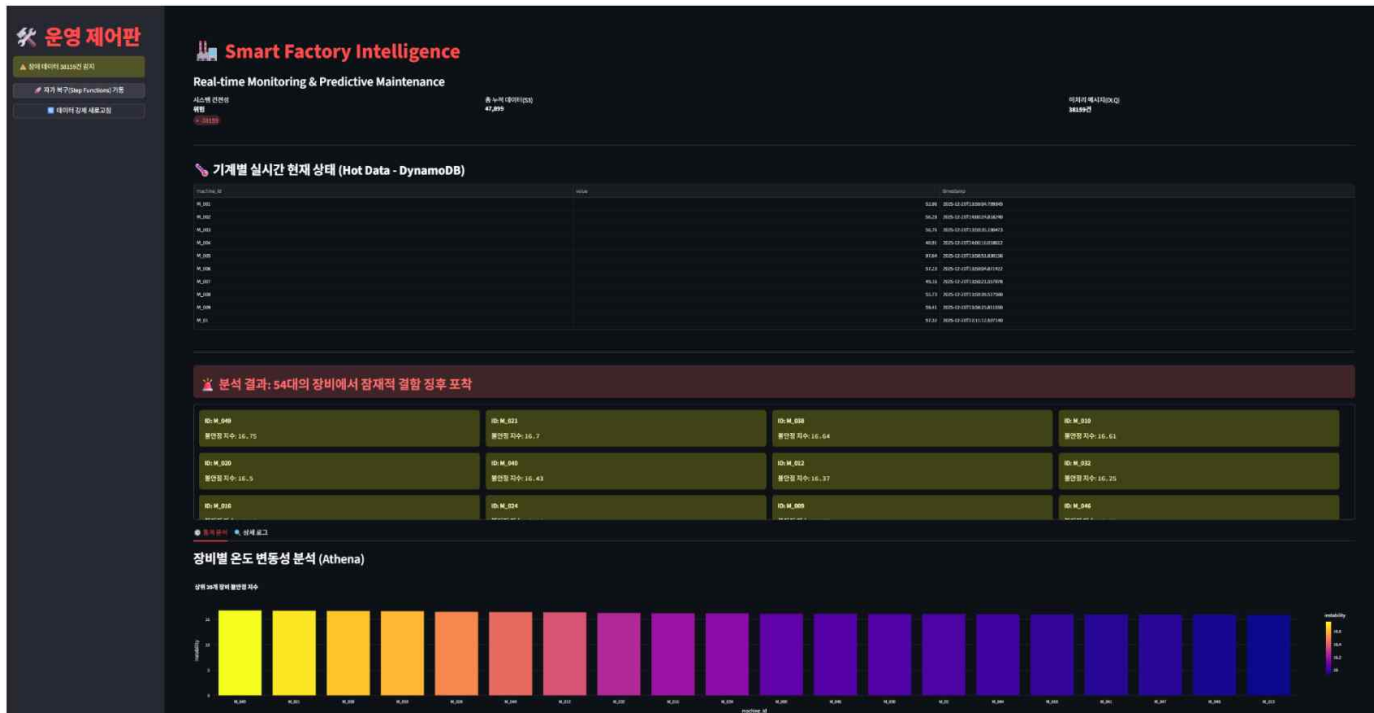
| #  | machine_id | instability | avg_value | record_count |
|----|------------|-------------|-----------|--------------|
| 1  | M_049      | 16.75       | 56.1      | 951          |
| 2  | M_021      | 16.7        | 56.35     | 953          |
| 3  | M_038      | 16.64       | 55.95     | 932          |
| 4  | M_010      | 16.61       | 55.81     | 965          |
| 5  | M_020      | 16.5        | 55.83     | 950          |
| 6  | M_040      | 16.43       | 55.55     | 929          |
| 7  | M_012      | 16.37       | 55.43     | 950          |
| 8  | M_032      | 16.25       | 55.47     | 932          |
| 9  | M_016      | 16.19       | 55.05     | 904          |
| 10 | M_024      | 16.16       | 55.47     | 959          |
| 11 | M_009      | 16.07       | 55.25     | 954          |
| 12 | M_046      | 16.05       | 55.13     | 994          |
| 13 | M_030      | 16.04       | 55.05     | 934          |
| 14 | M_02       | 16.03       | 55.86     | 10           |
| 15 | M_044      | 16.01       | 55.43     | 961          |
| 16 | M_018      | 15.99       | 55.12     | 984          |
| 17 | M_047      | 15.97       | 55.15     | 944          |

# cloudwatch / Streamlit

cloud watch를 이용해 인프라 전 계층을 모니터링 할 수 있게 대시보드를 생성하였습니다.



streamlit 을 이용해 전체적인 iot 센서 실시간 현황과 위험 징후를 보이는 센서들을 관리할 수 있는 대시보드를 만들었습니다





# 고도화

## 트래픽 폭증 및 처리 지연 발생 시 시스템 반응 분석

lambda코드에 잠깐 대기 로직 sleep(10)을 넣고, simulator에 데이터 보내는 간격을 0.1초로 매우 빠르게 수정 후 실행함

lambda\_function.py X

lambda\_function.py

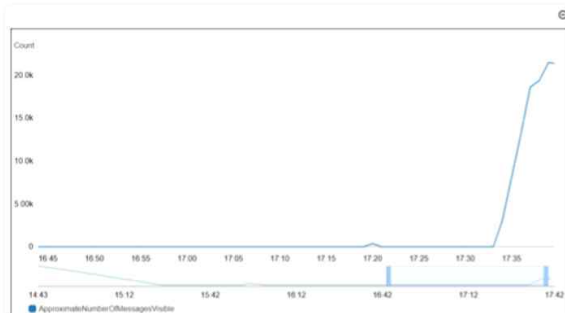
```
6  ss = boto3.client('s3')
7
8  TABLE_NAME = 'FactoryLatestState'
9  BUCKET_NAME = 'smart-factory-data-seohyeon-2025'
10
11 # Decimal 타입을 JSON으로 변환하기 위한 보조 함수
12 def decimal_default(obj):
13     if isinstance(obj, Decimal):
14         return float(obj)
15     raise TypeError
16
17 def lambda_handler(event, context):
18     # --- 명목 실험용 코드 시작 ---
19     print("데이터 처리 중... (인위적 지연 발생)")
20     time.sleep(10) # 2. 처리 시간을 10초로 강제 지연 (명목 유도)
21     # --- 명목 실험용 코드 끝 ---
22
```

```
NUM_DEVICES = 1000    # 기계 대수를 확장
INTERVAL = 0.01        # 전송 간격 (부하 증가 테스트 때 사용)
ERROR_RATE = 0.05      # 5% 확률로 고장난 데이터(DLQ용) 발송
ANOMALY_RATE = 0.1    # 10% 확률로 이상치(80도 이상) 발송
```

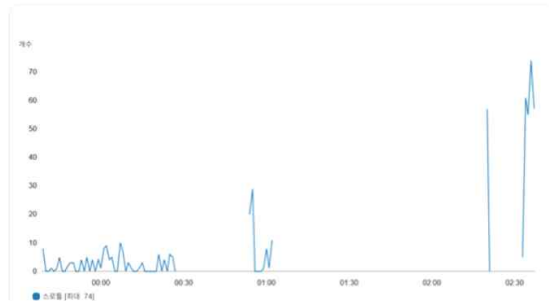
# 트래픽 폭증 및 처리 지연 발생 시 시스템 반응 분석

데이터 유입 속도가 처리 속도를 압도하며 SQS 대기열에 메시지가 급격히 적체됨.  
lambda에 쓰로틀링이 걸리는 것을 확인하였음.

표시된 메시지의 대략적 **연속** 재설정



스스로



지표에서 보기

닫기

지표에서 보기

닫기

## 대기열 (2)

검색: 접두어를 기준으로 대기열 검색

|                       | 이름                                 | 유형 | 생성 날짜                  | 사용 가능한 메시지 | 이동 중인 메시지 | 암호화                |
|-----------------------|------------------------------------|----|------------------------|------------|-----------|--------------------|
| <input type="radio"/> | <a href="#">factory-data-dlq</a>   | 표준 | 2025-12-20T20:42+09:00 | 55053      | 0         | Amazon SQS 키(SSE-S |
| <input type="radio"/> | <a href="#">factory-data-queue</a> | 표준 | 2025-12-20T20:44+09:00 | 19386      | 1180      | Amazon SQS 키(SSE-S |



## 트래픽 폭증 및 처리 지연 발생 시 시스템 반응 분석

SQS가 데이터 폭주 시 방어막 역할을 한다는 것을 보고 확장성을 알 수 있음. BUT Consumer Lambda 하나가 DB 조회, S3 저장, 이상 탐지 로직을 다 처리하다 보니 코드가 복잡해지고 관리 포인트가 늘어서 스로틀링이 걸리는 상황을 확인함

변경 전: Ingest Lambda → SQS → Consumer Lambda (DB 저장 + S3 저장)

변경 후: Ingest Lambda → SNS Topic

SNS Topic → SQS → Consumer Lambda (실시간 관제 역할 : 이상 탐지 후 dynamoDB 저장 전용)

SNS Topic → Storage Lambda (기록 보관 : S3 에 데이터 바로 저장 전용)

## 트래픽 폭증 및 처리 지연 발생 시 시스템 반응 분석

## ingest lambda

코드 소스 정보

Open in Visual Studio Code

에서 업로드

FactoryIngestFunction

lambda\_function.py

```

1 import json
2 import boto3
3 import os
4
5 sqs = boto3.client('sqs')
6 TOPIC_ARN = "arn:aws:sns:ap-northeast-2:923187443212:facto
7 def lambda_handler(event, context):
8     try:
9         # API Gateway로부터 받은 데이터 추출
10         body = json.loads(event['body'])
11
12         # SQS 대신 SNS로 데이터 발행 (Fan-out 시작점)
13         response = sns.publish(
14             TopicArn=TOPIC_ARN,
15             Message=json.dumps(body)
16         )
17
18     return {
19         'statusCode': 200,
20         'body': json.dumps({'message': 'Data queued su
21     }
22 except Exception as e:
23     return {
24         'statusCode': 500

```

EXPLORER

FACTORYINGESTFUNCTION

lambda\_function.py

DEPLOY Undeployed

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

TEST EVENTS [NONE SELECTED]

Create new test event

## 함수 개요 정보

다이어그램

템플릿



API 게이트웨이

+ 트리거 추가

Infrastru

+ 대상 추가

# 트래픽 폭증 및 처리 지연 발생 시 시스템 반응 분석

sns에서 sqs큐로 데이터 흘러가도록 구독 생성

factory-data-sns

편집

삭제

메시지 게시

## 세부 정보

이름

factory-data-sns

ARN

arn:aws:sns:ap-northeast-2:923187443212:factory-data-sns

표시 이름

-

유형

표준

주제 소유자

923187443212

< **구독** 액세스 정책 데이터 보호 정책 전송 정책(HTTP/S) 전송 상태 로깅 >

구독 (2)

편집

삭제

확인 요청

구독 확인

구독 생성

검색

< 1 > ⚙

| ID                                                       | 엔드포인트               | 상태    | 프로토콜  |
|----------------------------------------------------------|---------------------|-------|-------|
| <input type="radio"/> <a href="#">22af6909-7798-...</a>  | sh99429@naver.c...  | ✅ 확인됨 | EMAIL |
| <input type="radio"/> <a href="#">daea0783-c0a1-4...</a> | arn:aws:sqs:ap-n... | ✅ 확인됨 | SQS   |

consumer 람다는 이상치 탐지,  
DynamoDB에 저장하는 역할을 함  
(기존 consumer 람다 역할에서 s3저장 부분은  
storage 람다 만들어서 분리함)

## storage lambda 생성

sns에서 바로 데이터를 받아 s3에 원본 로그를 저장합니다.

코드 소스 정보

Open in Visual Studio Code

에서 업로드

storageLambda

EXPLORER

STORAGE...

index.py

index.py

```

12 def lambda_handler(event, context):
13     try:
14         # SNS로부터 전달받은 Records 반복 처리
15         for record in event['Records']:
16             # 1. SNS 메시지 추출 (SNS는 원본 데이터를 Message 필드에 담아 보냄)
17             sns_message = json.loads(record['Sns']['Message'])
18
19             # 2. 데이터에서 필요한 정보 추출
20             machine_id = sns_message.get('machine_id', 'unknown')
21             timestamp_str = sns_message.get('timestamp', datetime.now().isoformat())
22             event_id = sns_message.get('event_id', f'evt_{int(datetime.now().timestamp())}')
23
24             # 3. Athena 분석을 위한 S3 경로(Partition) 생성
25             # 예: raw/year=2025/month=12/day=21/machine_id/event_id.json
26             ts = datetime.fromisoformat(timestamp_str)
27             s3_key = (
28                 f"raw/year={ts.year}/month={ts.month:02d}/day={ts.day:02d}/"
29                 f"{machine_id}/{event_id}.json"
30             )
31

```

DEPLOY

Undeployed

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

문제가 있는 데이터는 dlq sqs로 이동합니다.

storageLambda

Layers

SQS

+ 대상 추가

구성

발행

버전

대상 정보

Search 대상

소스

이벤트 소스 매핑

조건

대상

비동기식 호출

실행 시

arn:aws:sqs:ap-northeast-2:923187443212:factory-data-dlq

1. 장애 격리: 이제 S3 저장 중 에러가 발생해도, Consumer Lambda 경로는 영향을 받지 않고 공장 상태를 계속해서 DynamoDB에 업데이트할 수 있습니다.
2. 병렬 처리: DB 저장과 S3 저장이 동시에 일어나므로, 전체 시스템의 응답 속도가 개선되었습니다.
3. 유지보수: 이상 탐지 로직을 수정할 때 S3 저장 코드를 건드릴 필요가 없어 관리 포인트가 명확해졌습니다.

# Fault Tolerance & DLQ

이전 방식 : dlq에 쌓인 데이터는 수동으로 리드라이브 해야 했음  
=> 비효율적

## 대기열 (2)

Q 점두사를 기준으로 대기열 검색

|                       | 이름                                 | 유형 | 생성 날짜                  | 사용 가능한 메시지 | 이동 중인 메시지 | 암호  |
|-----------------------|------------------------------------|----|------------------------|------------|-----------|-----|
| <input type="radio"/> | <a href="#">factory-data-dlg</a>   | 표준 | 2025-12-20T20:42+09:00 | 53927      | 60        | Ami |
| <input type="radio"/> | <a href="#">factory-data-queue</a> | 표준 | 2025-12-20T20:44+09:00 | 22336      | 277       | Ami |

Amazon SQS > 대기열 > factory-data-dlg > DLQ 리드라이브

Amazon SQS 대기열

**사용자 지정 대상으로 리드라이브**  
선택한 대상 대기열로 메시지 리드라이브

기존 대기열 선택  
arn:aws:sqs:ap-northeast-2:923187443212:factory-data-que...

**속도 제어 설정** | 정보

☒ 시스템 최적화  
초당 최대 메시지 수를 최적화된 SQS를 사용하여 메시지를 리드라이브합니다.

☐ 사용자 지정 최고 속도  
사용자 지정 초당 최고 메시지 속도를 리드라이브합니다. 허용 최고 속도는 초당 500 메시지입니다.

초당 메시지 수

**메시지 검사** | 선택 사항 | 정보

플링 설정 편집 | 플링 중지 | 메시지 플링

사용 가능한 메시지: 78736 | 플링 기간: 30 | 최대 메시지 수: 10 | 플링 진행 상황: 0%  
초당 0회 수신

메시지 (0) | 세부 정보 보기 | 삭제

Q 메시지 검색

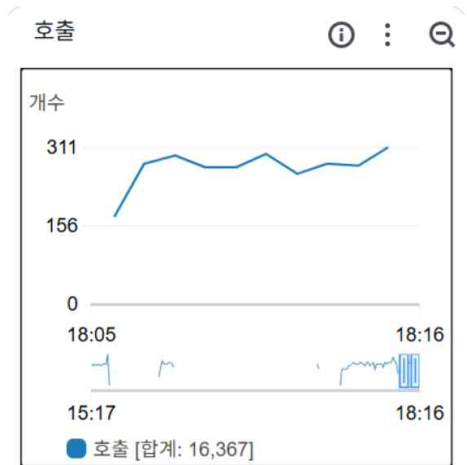
ID | 전송됨 | 크기 | 수신 수

메시지가 없습니다. 대기열에서 메시지를 내려면 메시지를 플링합니다.

메시지 플링

취소 | DLQ 리드라이브

리드라이브 버튼을 직접 눌렀을 때



ConsumerLamda의 호출 수가 확 늘어나는걸 볼 수 있음



DLQ의 수가 줄음

## 자동 복구 시스템 구현 - 복구 lambda 생성

### FactoryDataReplay

▼ 함수 개요

다이어그램

템플릿

+ 트리거 추가

+ 대상 추가

FactoryDataReplay

Layers (0)

설명

마지막 수정 12시간 전

함수 ARN `arn:aws:lambda:ap-northeast-2:923187443212:function:FactoryDataReplay`

함수 URL [정보](#)

Infrastructure Composer로 내보내기

다운로드 ▼

코드 테스트 모니터링 구성 별칭 버전

코드 소스

Open in Visual Studio Code

역사 업로드 ▼

EXPLORER

FactoryDataReplay

lambda\_function.py

DEPLOY

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

lambda\_function.py

```

14 def lambda_handler(event, context):
15     for msg in response['Messages']:
16         replay_count = int(attrs.get('replay_count', {}).get('StringValue', '0'))
17
18     if replay_count < 3:
19         # --- [Case A] SNS로 재투입 ---
20         new_count = replay_count + 1
21         sns.publish(
22             TopicArn=SMS_TOPIC_ARN, # 이제 SNS로 던져서 DB와 S3 양쪽 일괄 다 깨줍니다.
23             Message=body,           # 본문 데이터(JSON 문자열)
24             MessageAttributes={
25                 'replay_count': {'DataType': 'String', 'StringValue': str(new_count)}
26             }
27         )
28         print(f"SNS로 재발행 완료 (Fan-out 복구 {new_count}/3): {body}")
29     else:
30         # --- [Case B] 3회 이상: 최종 처리 (S3 저장 및 SNS 알림) ---
31         file_name = f'failed-data/msg-{context.aws_request_id}.json'
32         s3.put_object(Bucket=S3_BUCKET, Key=file_name, Body=body)
33
34         # 알림 전송
35         sns.publish(
36             TopicArn=SMS_TOPIC_ARN,

```

만약 3번 이상 dlq sqs에 들어간다면  
문제가 있는 모듈임을 확신하고  
위험 알림을 보내고 s3 버킷에 데이터를 따로 저장함

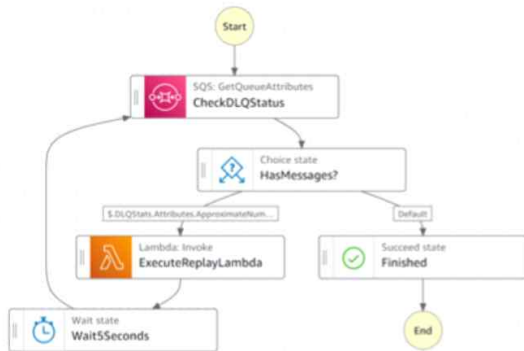


step functions 설정 : event bridge를 설정하여 dlq sqs에 데이터가 있으면 복구 람다를 실행하는 flow를 만들었다.

factory-data-fault-recovery-flow

실행 취소 다시 실행 확대 축소 중앙 중복 삭제

상태 추가



step functions를 주기적으로 자동 실행할 eventbridge 스케줄러 생성

일정 (1)

로딩된 일정 검색

모든 상태

모든 그룹

< 1 >

| <input type="checkbox"/> | 일정 이름                                           | 일정 그룹   | 상태 | 대상                               | 대상 유형              | 마지막으로 수정됨                          |
|--------------------------|-------------------------------------------------|---------|----|----------------------------------|--------------------|------------------------------------|
| <input type="checkbox"/> | <a href="#">Factory-Auto-Recovery-Scheduler</a> | default | 활성 | factory-data-fault-recovery-flow | SFN_StartExecution | Dec 20, 2025, 20:16:03 (UTC+00:00) |

잘 실행되는걸 확인할 수 있다

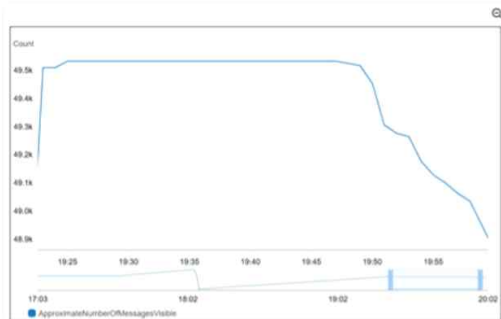
| <input type="checkbox"/> | 이름                                                   | 상태 | 시작 시간(로컬)                | 종료 시간(로컬)                | 기간 정보        |
|--------------------------|------------------------------------------------------|----|--------------------------|--------------------------|--------------|
| <input type="checkbox"/> | <a href="#">f7694704-03d5-4a05-909b-7ae7e96cd736</a> | 성공 | 2025년 12월 21일 5시 17분 38초 | 2025년 12월 21일 5시 38분 34초 | 00:20:56.010 |

eventbridge를 통해 step functions가 실행됨

-> receive\_message가 규격상 한번에 가져올 수 있는 메시지 최대치 10개 이기 때문에 느리지만 이동하는 것을 볼 수 있다.

dlq가 실시간으로 줄어드는 것을 확인할 수 있음

표시된 메시지의 대략적

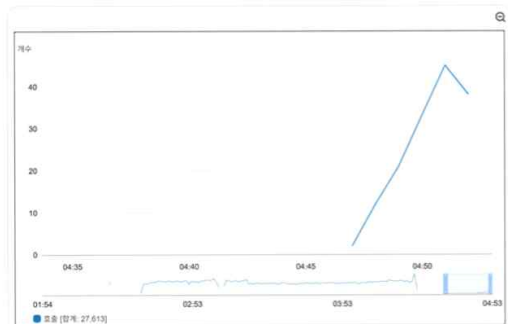


자료에서 보기

닫기

람다 호출

호출 확대/축소 재생



자료에서 보기

닫기

Step Functions > State machines > factory-data-fault-recovery-flow > Execution: a3a4a1c5-...

그래프 보기



작업 ▼

▶ 실행 중 ▶ 실패 ▶ 포착된 예외 ▶ 취소됨 ▶ 성공

State view

이벤트 (252)

속성으로 필터링 표시 날짜 및 시간 범위를 기준으로 필터링

< 1 2 >

| ID ▲ | 유형               | Step           | Resource                       | 다음 후에 시작됨    |
|------|------------------|----------------|--------------------------------|--------------|
| ▶ 1  | ExecutionStarted |                |                                | 0            |
| ▶ 2  | TaskStateEntered | CheckDLQStatus |                                | 00:00:00.037 |
| ▶ 3  | TaskScheduled    | CheckDLQStatus | aws-sdk-sqs:getQueueAttributes | 00:00:00.037 |
| ▶ 4  | TaskStarted      | CheckDLQStatus | aws-sdk-sqs:getQueueAttributes | 00:00:00.063 |

1. 데이터 유실 no: 에러가 나도 DLQ에 데이터가 보관되고, 복구 램다가 SNS로 다시 싸주니까 결과적으로 DB와 S3 양쪽에 데이터가 하나도 안 빠지고 잘 저장되는 걸 확인했습니다.
2. 수동 복구 자동화: 예전에는 실패한 데이터를 일일이 수동으로 옮겼는데, 이제는 Step Functions가 정해진 시간에 맞춰 자동으로 복구해주니까 운영이 훨씬 편해졌습니다.
3. 에러 관리: 계속 실패하는 데이터는 따로 S3에 격리하고 알림을 보내게 만들어서, 어떤 데이터가 왜 문제인지 나중에 따로 분석할 수 있는 환경까지 만들었습니다.



# 느낀점

## 1. 회복력 있는 시스템의 중요성

처음에는 에러가 아예 안나는 구조만 생각했는데, 실제 트래픽을 넣어보니 장애는 언제든지 날 수 있다는 걸 알았습니다.

에러를 막는 것보다 스스로 복구하는 탄력적 아키텍처가 왜 클라우드의 핵심인지 몸소 체감했습니다.

## 2. 아키텍처 수정 시 데이터 정합성

단순히 SQS로 복구하면 한쪽 경로(S3)가 누락된다는 점을 발견했을 때가 가장 큰 위기였습니다.

하지만 복구 지점을 SNS Topic으로 상향 조정하면서 서비스 간 결합도를 낮추고 데이터 정합성을 완벽하게 맞췄을 때 가장 큰 성취감을 느꼈습니다.

## 3. 현실적인 제약 안에서의 최적화 전략

Freetier 계정과 학생 계정을 병행해서 사용했는데 비용 문제와 접근 문제 때문에 Kinesis 대신 SQS를 선택했지만

오히려 이 과정에서 각 서비스의 비용 대비 성능을 꼼꼼히 따져보는 경험을 할 수 있었습니다

## 4. 앞으로의 계획

이번에 구축한 파이프라인에 SageMaker 같은 AI 모델을 붙여서 단순 통계를 넘어선 ai 기반 예지 보전 시스템으로 발전시켜 보고 싶습니다.



 클라우드컴퓨팅

---

# 감사합니다